CYBERSECURITY | AIORI-3

PS-013: Incident Timeline and Evidence Collection for Small Teams

A lightweight digital forensic tool: hash on collection, normalise to UTC, build a searchable timeline

Team NovaNexus | Sameer Saxena, Palak Saxena | Mentor: Anas Khan

Problem and Our Goal

The problem

- Logs are scattered across Linux hosts, web apps and network devices
- Clocks are unaligned and time zones differ
- Solo defenders rebuild the attack timeline by hand: slow and error-prone
- Evidence can be altered if not hashed at collection

Our goal

- Collect logs and preserve originals untouched

Normalise timestamps to UTC
Create one searchable chronological timeline
Export a summary incident report
Evaluate triage time against a manual baseline

System Architecture: 6-Stage Pipeline

1. Collect

Read raw log files (read-only copy)

2. Hash

SHA-256 of each file + manifest

3. Parse

Format-specific parsers to common fields

4. Normalise

Timezone + clock-skew fix to UTC

5. Timeline

Store in SQLite, search and filter

6. Report

Export HTML/PDF summary

Original evidence stays in a read-only /evidence folder. All processing works on normalised copies. No live endpoint takeover or malware removal.

Tech Stack and Modules

Module	Tool / Approach
Ingestion + parsing	Python 3, regex parsers (one class per log format)
Hashing	hashlib SHA-256, manifest.json with hash + time + size
Timestamp normalisation	datetime + zoneinfo, per-source offset and skew config
Storage	SQLite: original_events and normalised_events tables
Timeline viewer	Streamlit web UI: search, filter, sort by UTC
Report export	Jinja2 HTML template to PDF/Markdown

How We Meet the Key Requirements

- 2 log formats: Linux auth.log (syslog) and Nginx/Apache access log
- Evidence integrity: SHA-256 at collection, re-verify before reporting
- Originals preserved; normalised copy stored separately
- Time zones + clock skew: parse offset, apply per-source correction, log it
- Parsing gaps: unparsed lines saved to a gap report with line numbers and reason
- Search: keyword, IP, user, time range on the UTC timeline

Test Scenarios and Evaluation

Scenario 1

SSH brute force then successful login

Scenario 2

Web attack: SQL injection / path scan on web app

Scenario 3

Privilege escalation with clock skew between hosts

Evaluation Plan

- Parsing coverage: parsed lines / total lines, with gaps documented

Triage time: compare manual investigation with the tool for each scenario
Record results only after running the test datasets

Validation Plan (Results Pending)

Scenario	Lines	Parsed	Gaps	Coverage	Tool time
1: SSH brute force	Pending	Pending	Pending	Pending	Pending
2: Web attack	Pending	Pending	Pending	Pending	Pending
3: Clock skew	Pending	Pending	Pending	Pending	Pending

- Planned checks:

Verify SHA-256 hashes before and after processing to check evidence integrity
Test timezone conversion and configured clock-skew correction using known sample timestamps
Record unparsed lines with their line numbers and reasons
Measure parsing coverage and triage time for all three scenarios; add numerical results after testing

Pseudocode

```
FOR each log file:
    COPY to read-only evidence/ ; hash = SHA256(file)
    SAVE (file, hash, size, time) to manifest.json
FOR each line in evidence file:
    event = PARSE(line, format)
    IF event is NULL: gaps.ADD(file, line_no, reason)
    ELSE:
        event.utc = TO_UTC(event.time, tz, skew)
        events.ADD(event)
SORT events BY utc
STORE original + normalised copies IN SQLite
VERIFY all file hashes == manifest
EXPORT report(summary, key events, gaps, hashes)
```

Deliverables and Demo Plan

- **GitHub repository:** Create the team repository from the official AIORI template; follow the portal's collaboration and ownership-transfer instructions.

Presentation: Upload the final presentation as a PDF using the structure specified by the hackathon portal.

Proposed workflow: Log collection, evidence hashing, timestamp normalization, timeline search, and report generation.

Evaluation plan: Test on three incident scenarios and report parsing coverage, evidence-integrity checks, and triage-time measurements only after testing.

Demo or deliverables: Provide a live demo link if available. If the solution cannot be demonstrated live, upload the final deliverable files for evaluator review.

Reference: NIST SP 800-86: Guide to Integrating Forensic Techniques into Incident Response.